Ez a dokumentum Markdown formátumban is elérhető a repóban:
github.com/c9n6zk/DBMate/blob/master/docs/SUBMISSION.md
A screenshotok: github.com/c9n6zk/DBMate/tree/master/docs/screenshots

DBMate — LLM-alapú adatbázis-asszisztens

Feladat azonosítója: Prompt Engineering — Házi feladat **Feladat címe:** Egy LLM-alapú adatbázis-asszisztens webalkalmazás fejlesztése **Beadó:** Jáky Dániel Fülöp **Dátum:** 2026-03-24 **Git repository:** https://github.com/c9n6zk/DBMate

---

1. A feladat leírása

Egy LLM-alapú adatbázis-asszisztens webalkalmazás fejlesztése, amely képes SQL adatbázis-sémák importálására és elemzésére. Az alkalmazás természetes nyelvű lekérdezéseket SQL-re alakít, valamint javaslatokat tesz indexelésre, teljesítményoptimalizálásra és az adatbázis normalizálására. A javasolt módosítások automatikusan alkalmazhatók, és a módosított sémák exportálhatók.

2. A megoldás körülményei

A fejlesztés teljes egészében **Claude Code** (Anthropic CLI) segítségével történt, prompt engineering technikákat alkalmazva a tervezéstől a kódgenerálásig. A projekt ~100 TypeScript/TSX forrásfájlból áll, amelyeket fázisonként, iteratívan készítettem el.

A fejlesztési folyamat három fő lépésre tagolódott:

1. **Tervezés** — Részletes Master Plan dokumentum (~2800 sor) készítése, amely definiálta az architektúrát, az API végpontokat, az adatbázissémát és az UI wireframe-eket. Ez lett a „kontextus ablak", amelyre a fejlesztés során minden prompt hivatkozott.

2. **Fázisonkénti implementáció** — 6 fejlesztési fázisban készült el a teljes alkalmazás (Import → Dashboard → Optimizer → Migrations → Seed → Export), minden fázisban a korábbi kód kontextusára építve. Az AI-modell konzisztens kódot generált, mert a Master Plan állandó referenciát biztosított.

3. **AI Improvement Plan** — Egy önálló review ciklus, ahol az AI saját kódját elemezte és 52 javítási pontot azonosított (v6-ig iterálva). Ezek közül a kritikusakat implementáltuk:

centralizált JSON parsing (`extractJSON`), retry+Zod validáció, few-shot prompting, hibrid statikus+AI elemzés.

API elérés: Claude Max proxy vs. API kulcs

A fejlesztés során **nem rendelkeztünk Anthropic API kulccsal** — ehelyett a **Claude Max előfizetés** (claude.ai webes felület) által biztosított tokent használtuk. Ehhez egy saját **OAuth proxy szervert** fejlesztettünk (Node.js, külön repository: `claude-code-proxy`), amely a claude.ai session tokent felhasználva közvetíti az API hívásokat a lokális Next.js szerver felé.

Ez komoly kihívást jelentett, mert a proxy **nem támogatja az Anthropic `tool_use` API-t** (structured output) — csak szöveges (`text`) válaszokat tud visszaadni. Így az AI válaszokból saját JSON-parserrel kellett kinyerni a strukturált adatot.

Ennek megoldására **kettős stratégiát** dolgoztunk ki:

- **Proxy mód** (Claude Max) — Az AI szöveges válaszából a `extractJSON()` függvény robusztusan kinyeri a JSON-t (markdown code block kezelés, csonkolt JSON javítás, brace-balancing), majd Zod sémával validáljuk.
- **Közvetlen API mód** (Anthropic API kulcs) — Ha a felhasználónak van API kulcsa, az Anthropic SDK közvetlenül csatlakozik a hivatalos API-hoz (nincs proxy köztes réteg). Az output parsing azonos (`extractJSON` + Zod), de nincs szükség a proxy szerver futtatására.

A mód automatikus detekciója az `isProxyMode()` függvénnyel történik (ellenőrzi, hogy az `ANTHROPIC_BASE_URL` környezeti változó be van-e állítva).

Az alkalmazás tehát mindkét módon működik:

- `.env.local` -ban `ANTHROPIC_API_KEY=sk-ant-...` → közvetlen API mód
- `.env.local` -ban `ANTHROPIC_BASE_URL=http://localhost:42069` → proxy mód (Claude Max előfizetéssel)

3. Technológiai stack

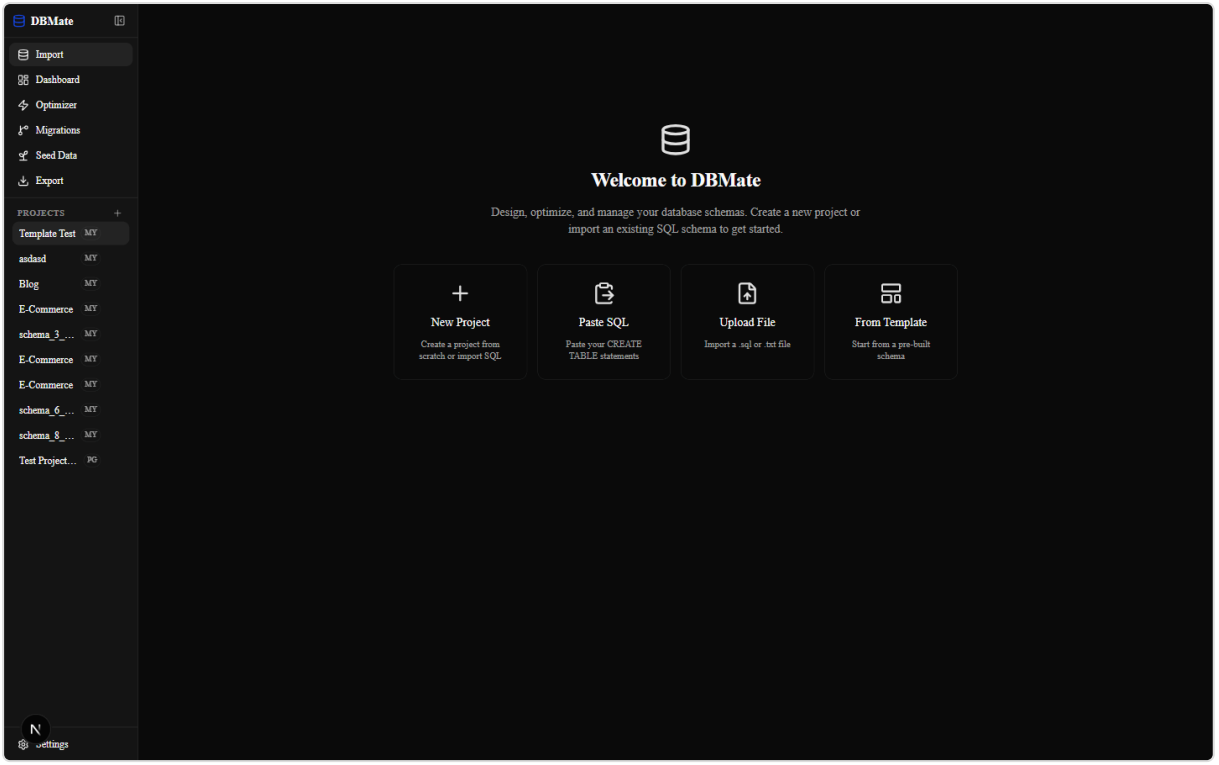
Technológia	Verzió	Szerep
Next.js	16.1.6	Full-stack keretrendszer (App Router, API Routes)
React	19	UI könyvtár
TypeScript	5.x	Típusbiztonság
TailwindCSS	4	Stíluskezelés
Zustand	5.0	Állapotkezelés (3 store)

Technológia	Verzió	Szerep
Zod	v4	Validáció (API határok)
better-sqlite3	12.6	Lokális SQLite adatbázis
Anthropic SDK	0.78	Claude API kliens
@xyflow/react	12.10	ER diagram vizualizáció
CodeMirror 6	6.0	SQL szerkesztő
node-sql-parser	5.4	SQL elemzés
Vitest	4.1	Unit teszt keretrendszer
Testing Library	16.3	React komponens tesztelés
Playwright	1.52	E2E tesztelés
axe-core	4.10	Akadálymentességi (a11y) tesztek

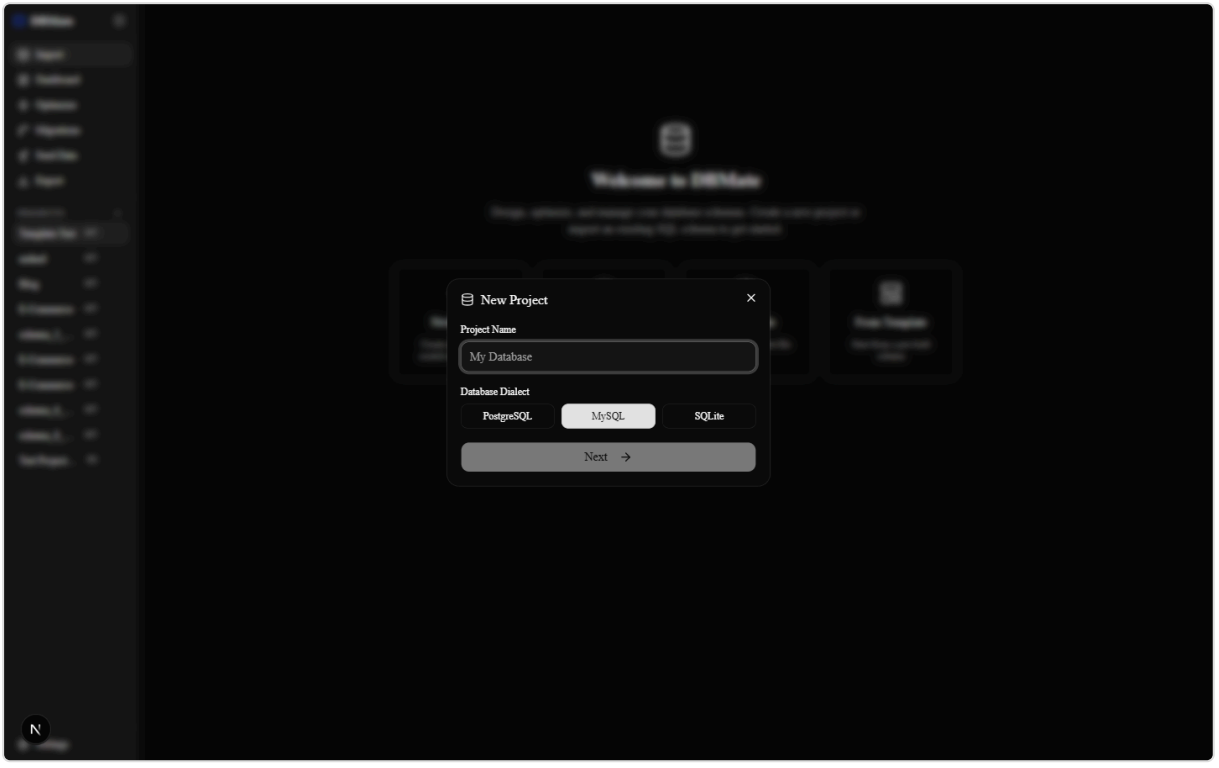
4. Funkciók és screenshotok

4.1. Import és projektkezelés

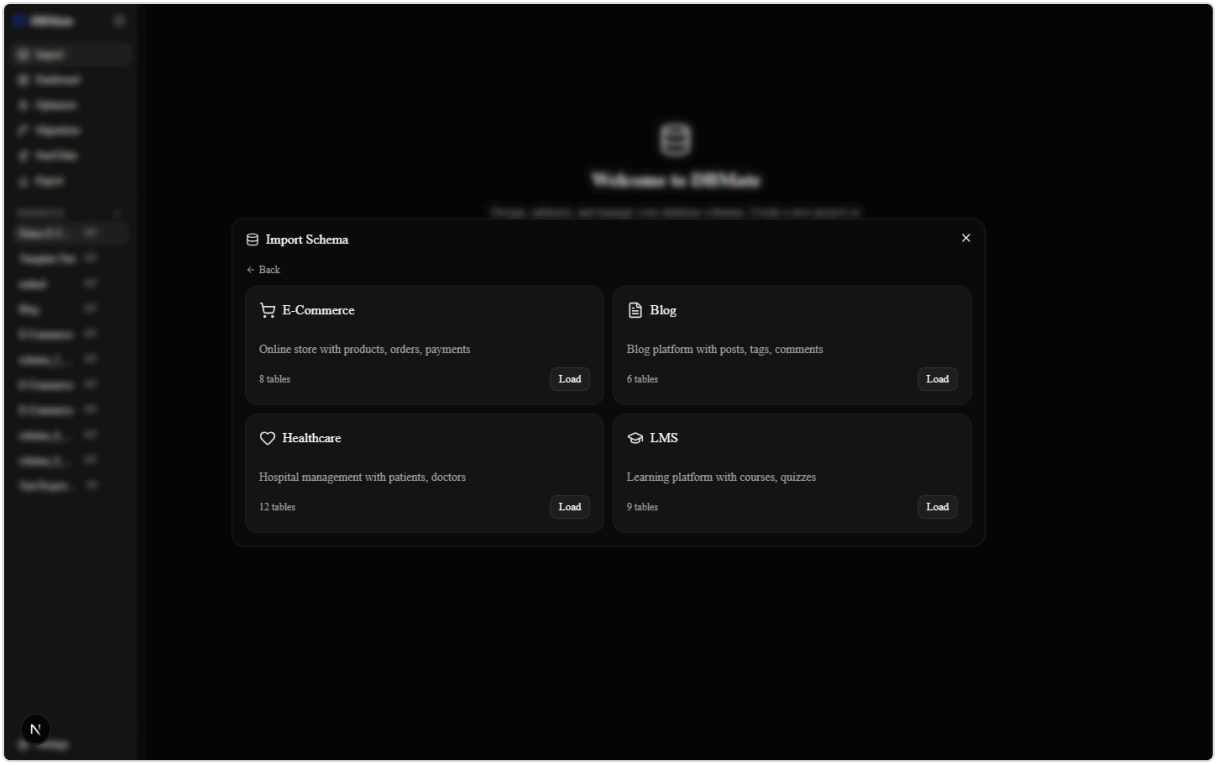
Az alkalmazás többféle módot kínál séma importálásra: új projekt létrehozása, SQL beillesztése, fájl feltöltés, vagy előre definiált template kiválasztása (E-Commerce, Blog, Healthcare, LMS).



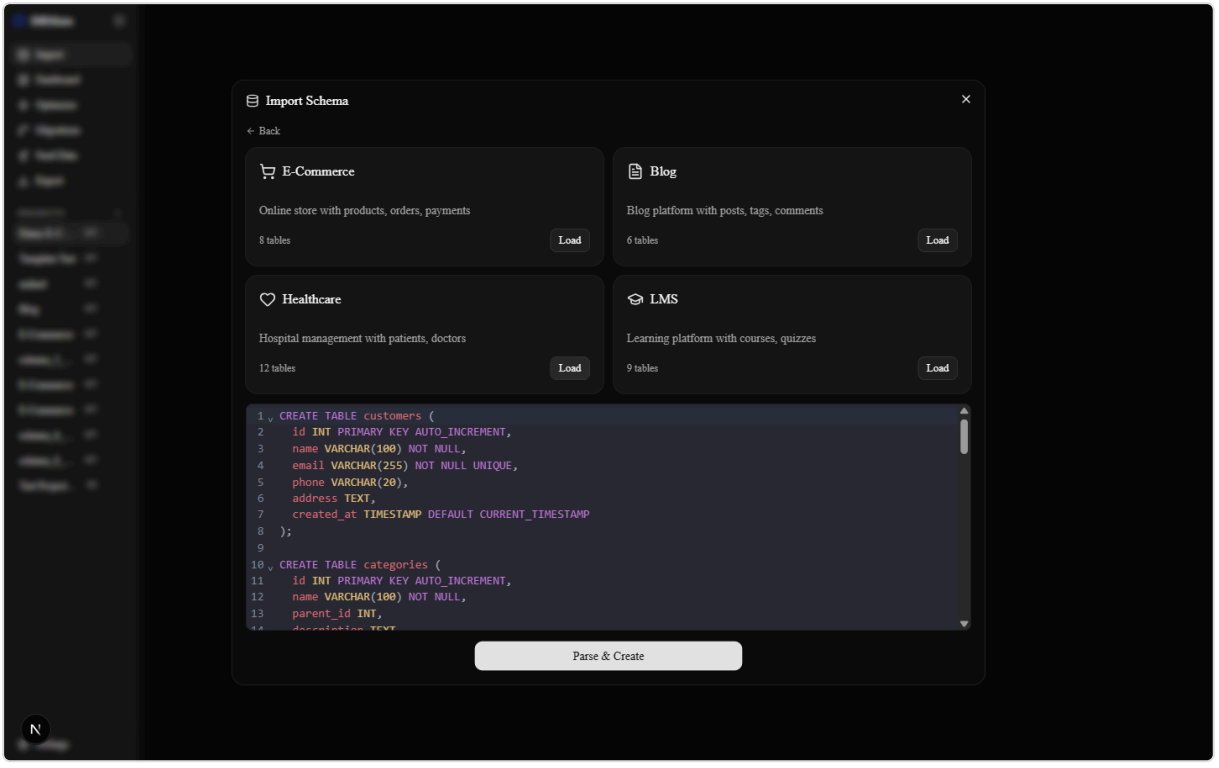
Welcome oldal — 4 import opció, sidebar projektlista



Új projekt létrehozása — név, dialektus választás (PostgreSQL/MySQL/SQLite)



Template galéria — előre definiált sémák egy kattintással betölthetők

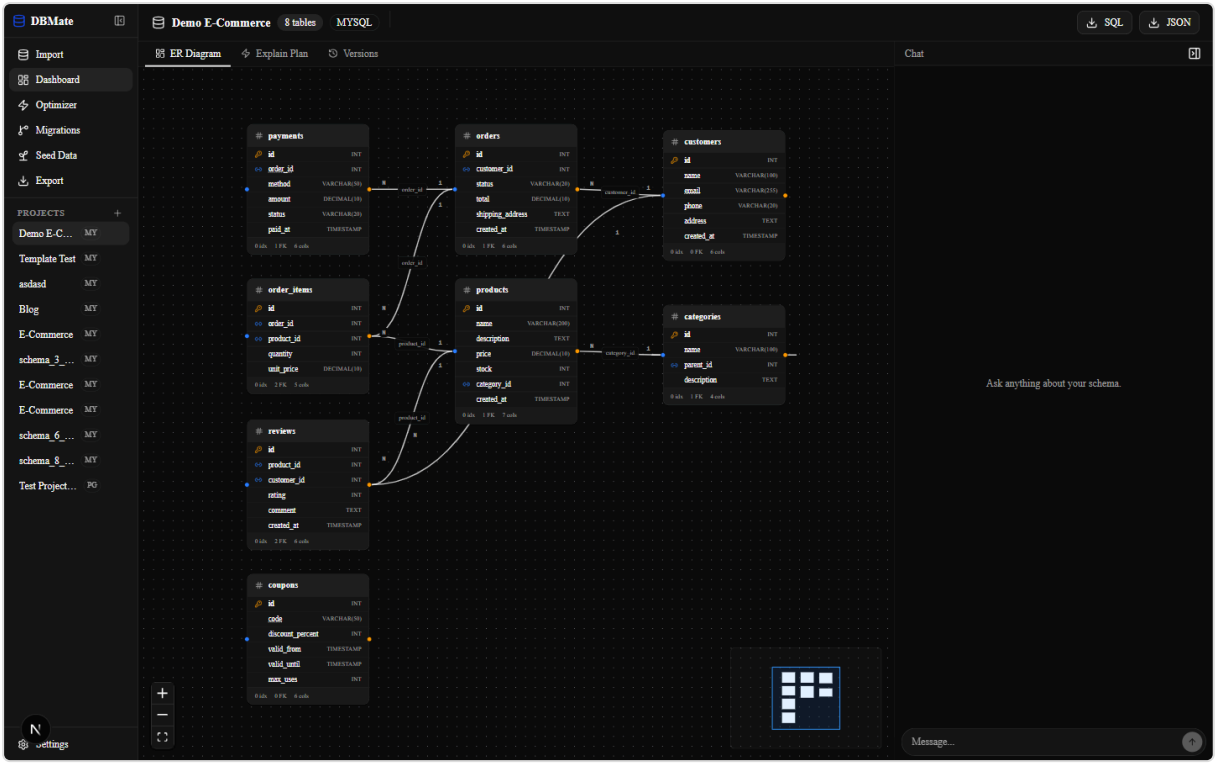


SQL szerkesztő — CodeMirror 6 syntax highlighting, Parse & Create gomb

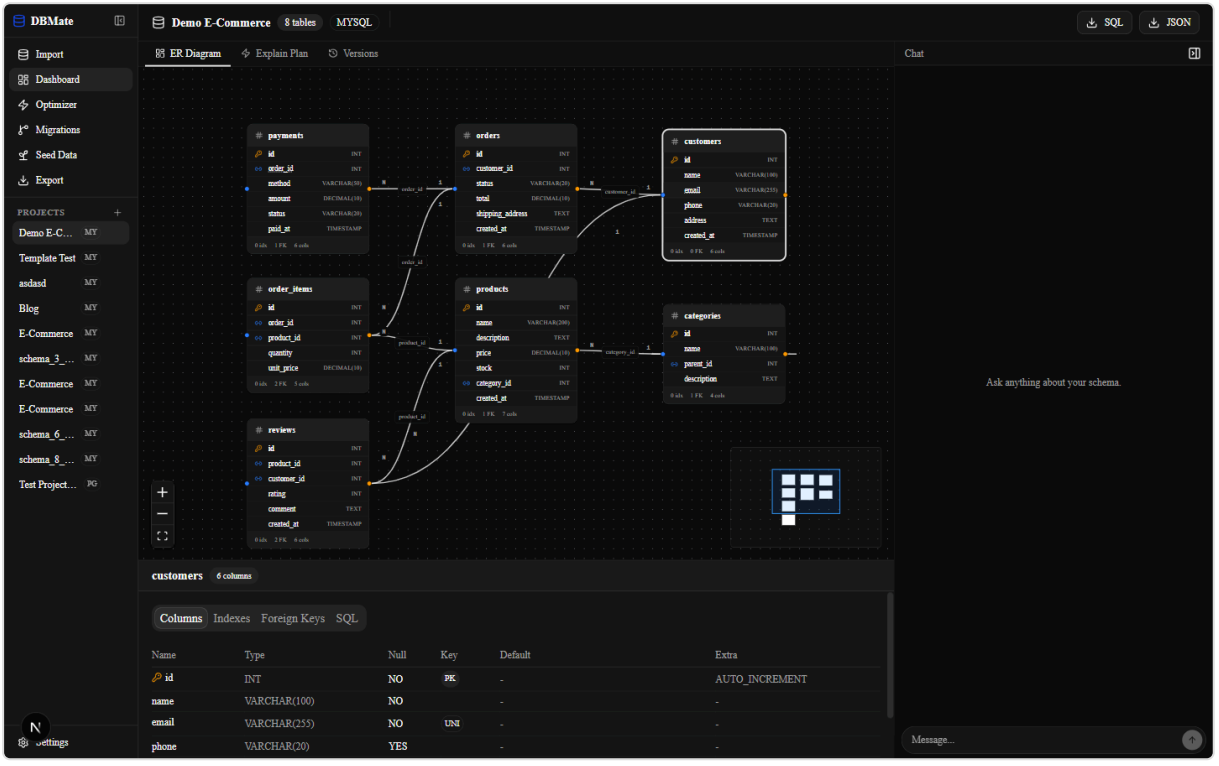
4.2. Dashboard — ER Diagram és séma nézet

A dashboard interaktív ER diagramot jelenít meg a @xyflow/react könyvtárral. A táblák húzhatók, a kapcsolatok (1:N, N:M) vizuálisan megjelennek. Egy táblára kattintva a részletes oszlop-

információk jelennek meg.



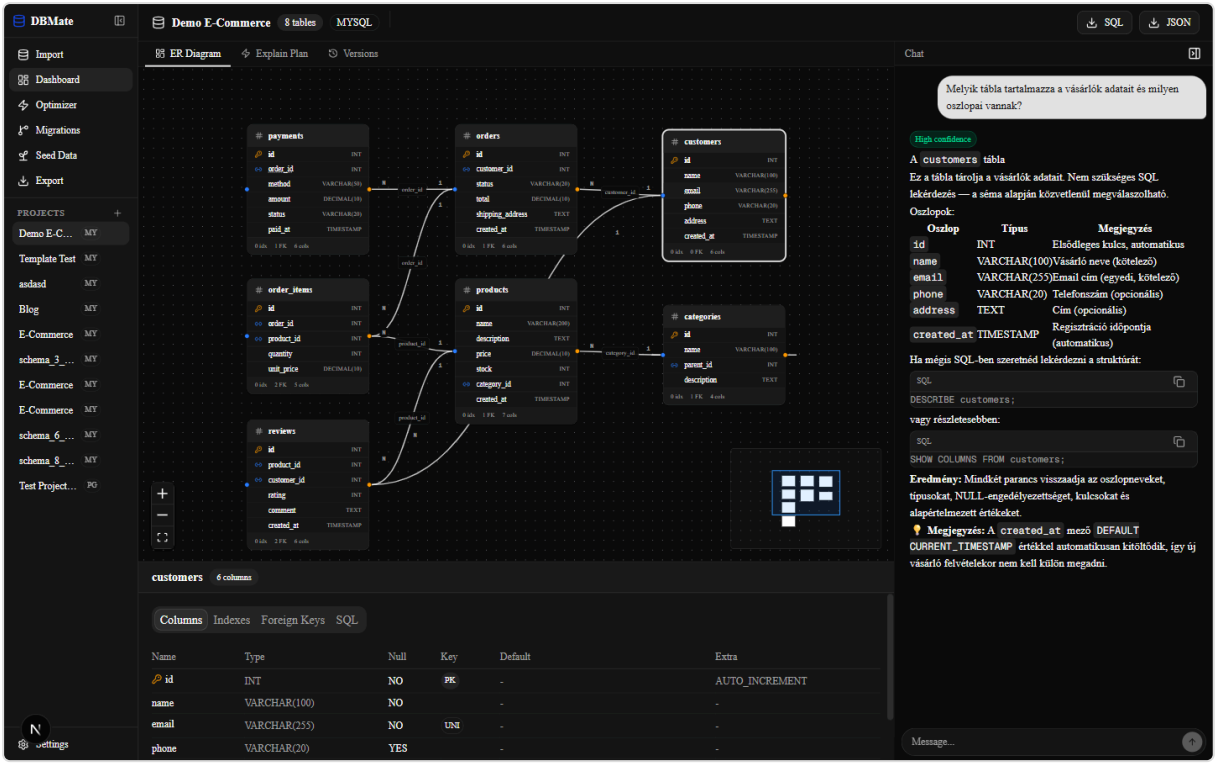
Interaktív ER diagram — 8 tábla, FK kapcsolatok, oszloptípusok



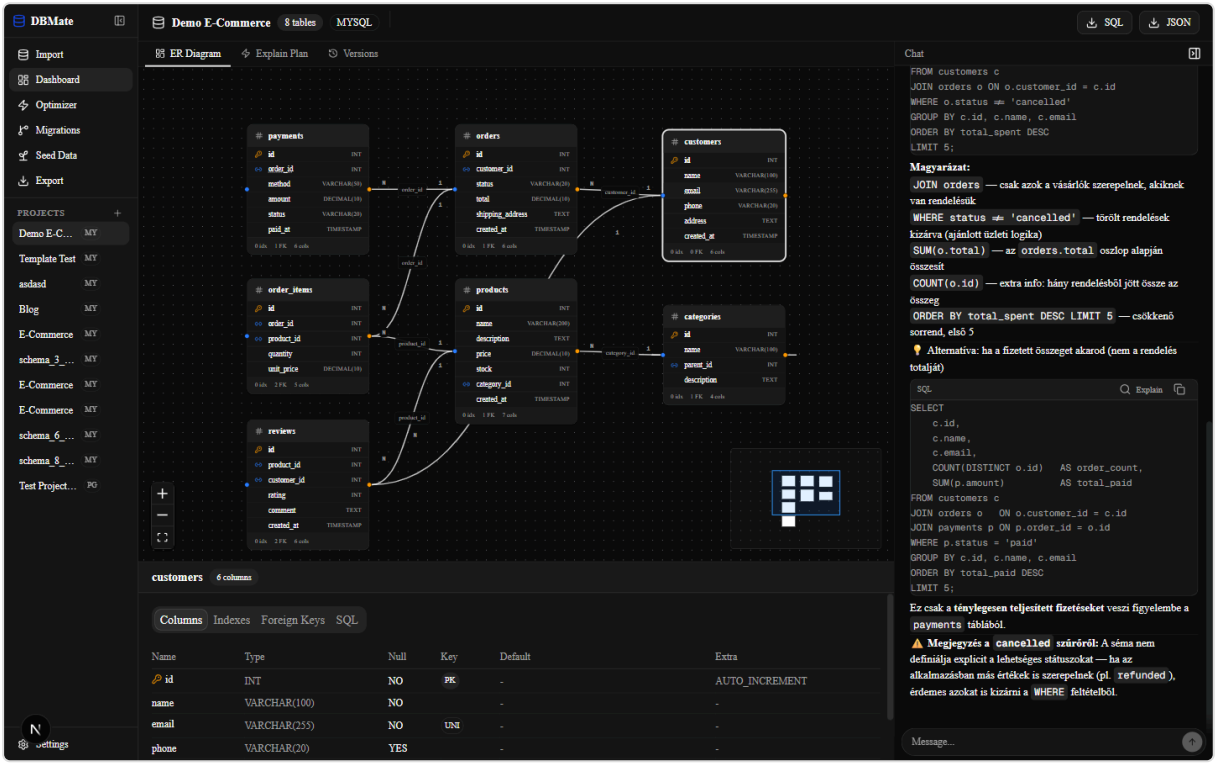
Tábla részletek — Columns, Indexes, Foreign Keys, SQL tab-ok

4.3. AI Chat — Természetes nyelv → SQL

A chat komponens természetes nyelvű kérdésekre válaszol a séma kontextusában. A válaszok tartalmazzák a generált SQL-t, magyarázatot, és egy confidence badge-et (High/Medium/Low). Az AI streaming SSE-vel válaszol.



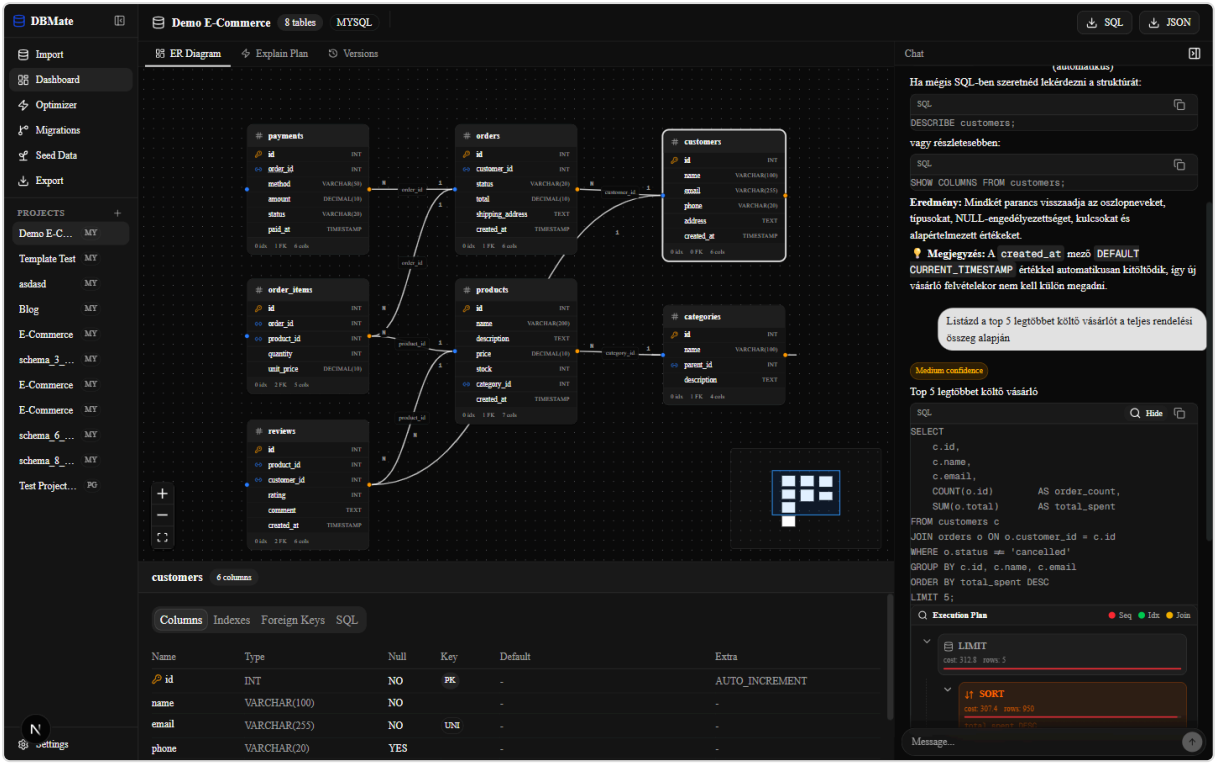
AI chat — magyar nyelvű válasz, "High confidence" badge, táblázatos formátum



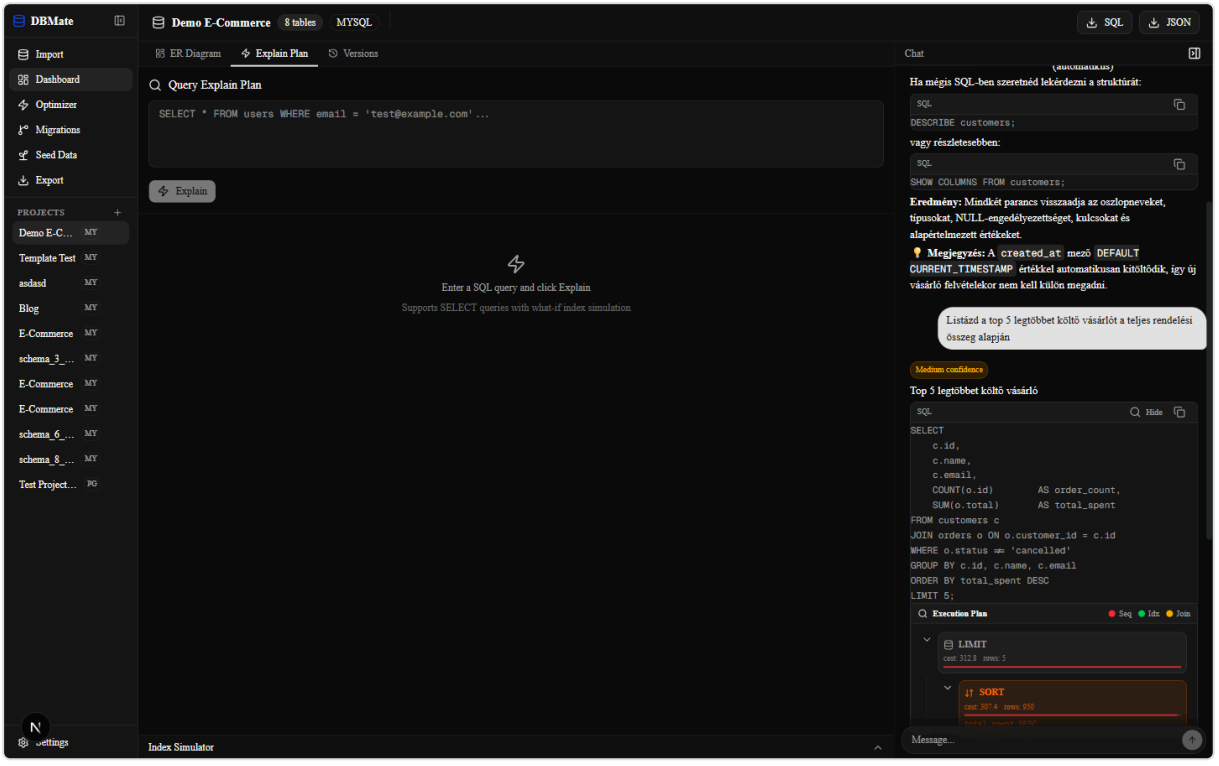
NL→SQL — komplex lekérdezés generálása (JOIN, GROUP BY, ORDER BY), Explain gomb

4.4. Explain Plan vizualizáció

A generált SQL-hez az AI végrehajtási tervet készít, amely fa-struktúrában jelenik meg (LIMIT → SORT → AGGREGATE → HASH JOIN → SEQ SCAN). Költségbecslés és sorlétszám minden lépésnél.



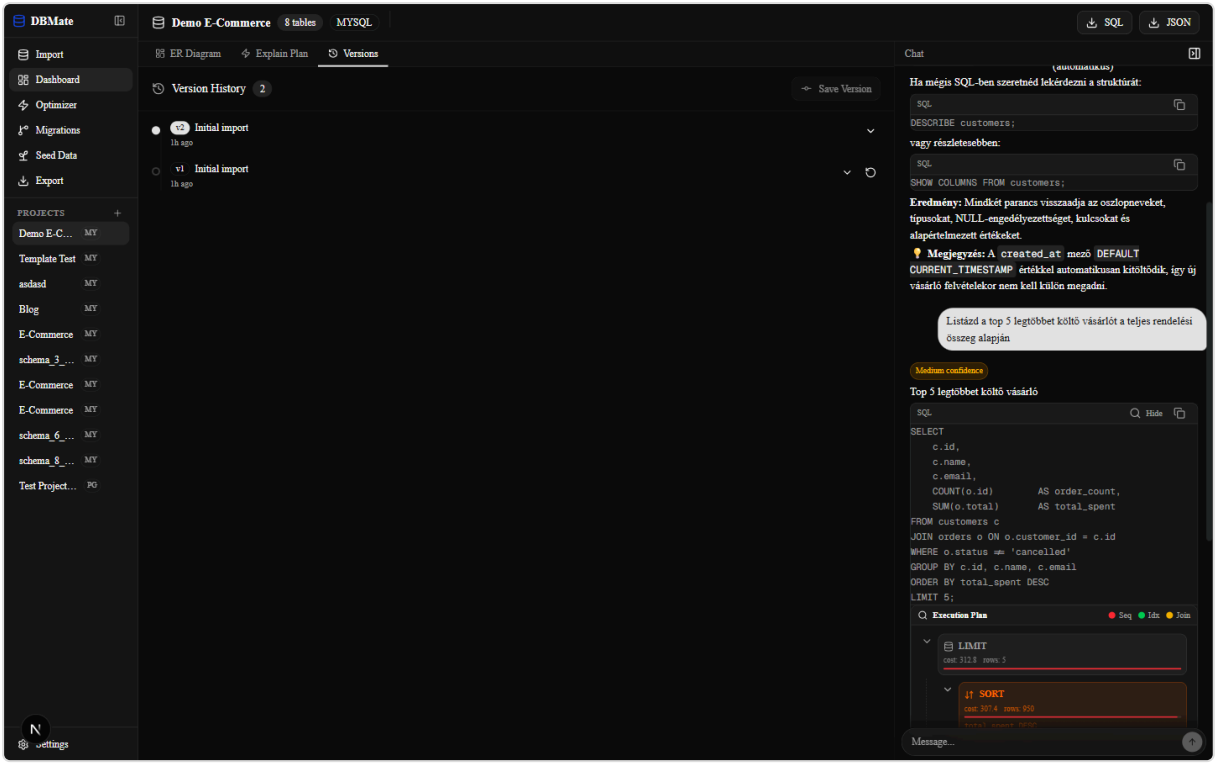
Explain Plan fa — végrehajtási terv vizualizáció a chat-ben



Explain Plan tab — külön nézet what-if index szimulációval

4.5. Version History

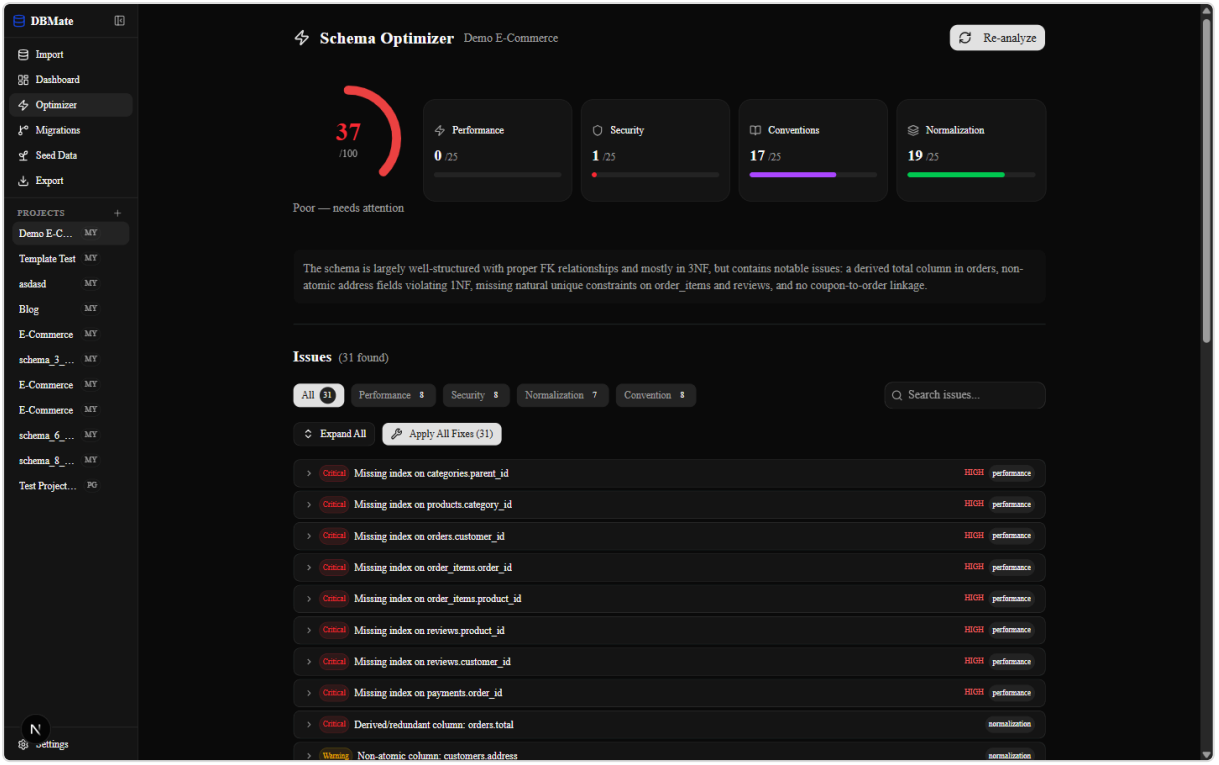
A séma minden módosítása verzióként mentődik. A timeline nézetben visszaállítható bármely korábbi verzió, és diff-összehasonlítás is elérhető.



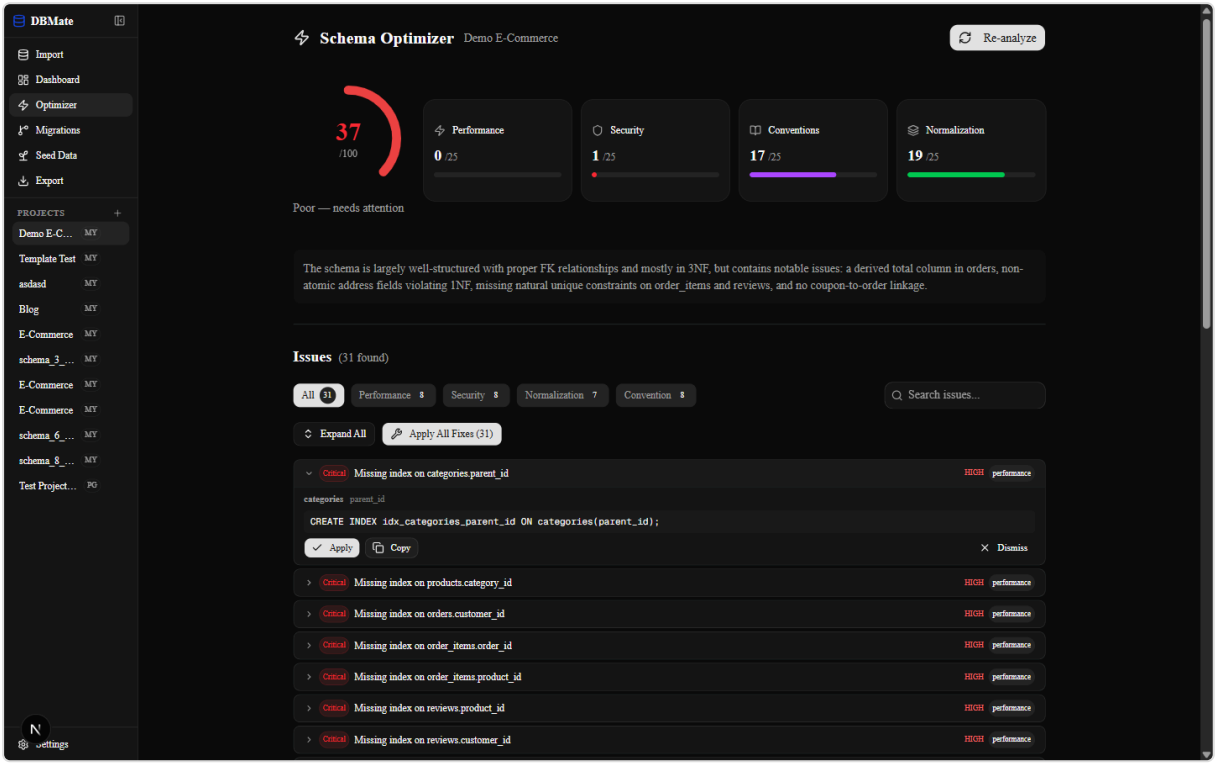
Version History — timeline, v1/v2, Show diff, Restore gombok

4.6. Schema Optimizer

A hibrid (statikus + AI) elemző 4 kategóriában pontozza a sémát: Performance, Security, Conventions, Normalization. Az összesített health score 0-100 között mozog. Minden issue-hoz automatikus fix SQL javaslat tartozik, amely egy kattintással alkalmazható.



Optimizer — Health Score 37/100, 31 issue, kategóriánkénti bontás

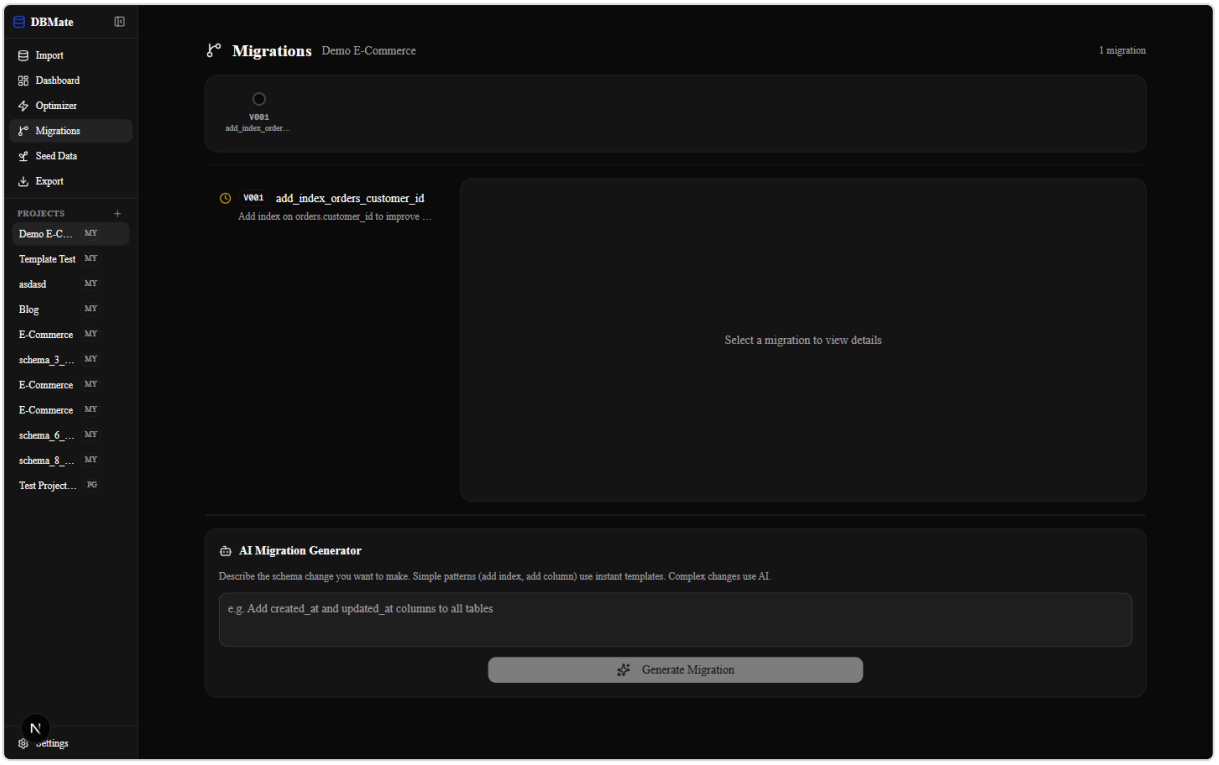


Issue részletek — CREATE INDEX javaslat, Apply/Copy/Dismiss gombok

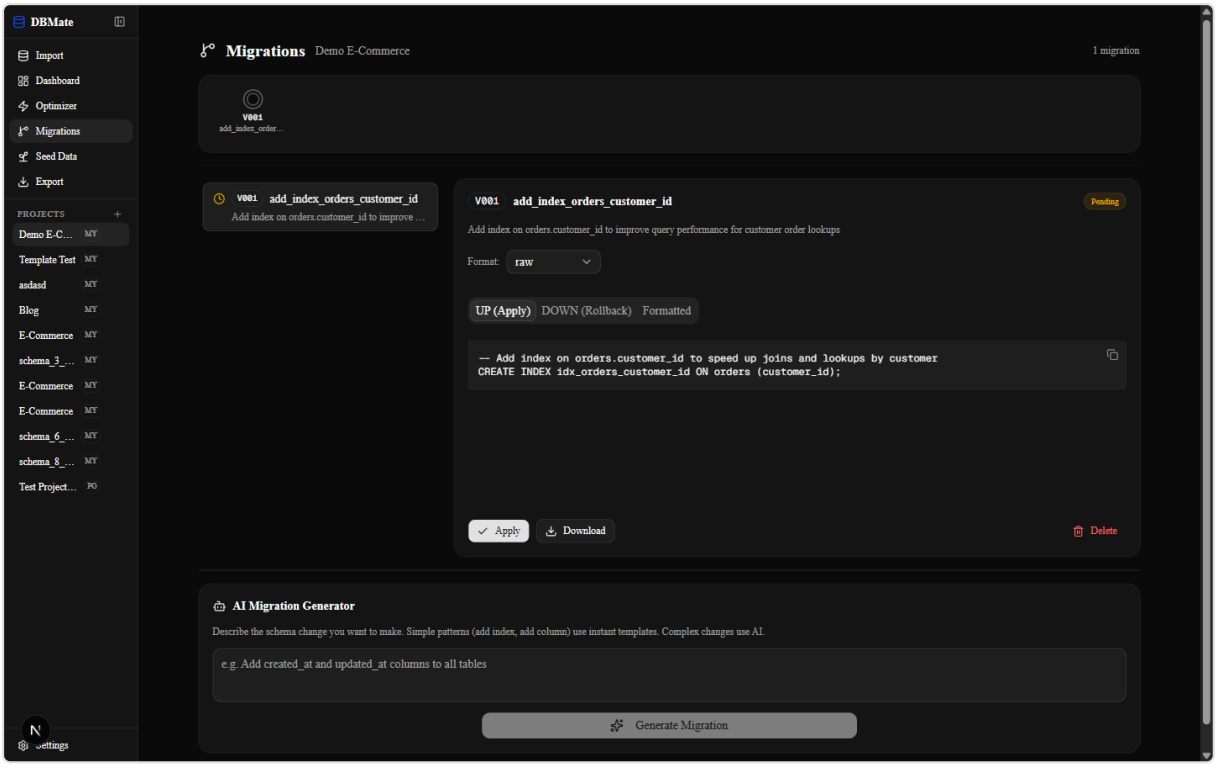
4.7. Migration Generator

A migrációs rendszer kettős stratégiával működik: egyszerű minták (index hozzáadás, oszlop hozzáadás) azonnali template-ekből generálódnak, komplex változások AI-t használnak. Minden

migráció tartalmaz UP és DOWN scriptet, formátum-választási lehetőséggel (Raw, Flyway, Liquibase, Prisma).



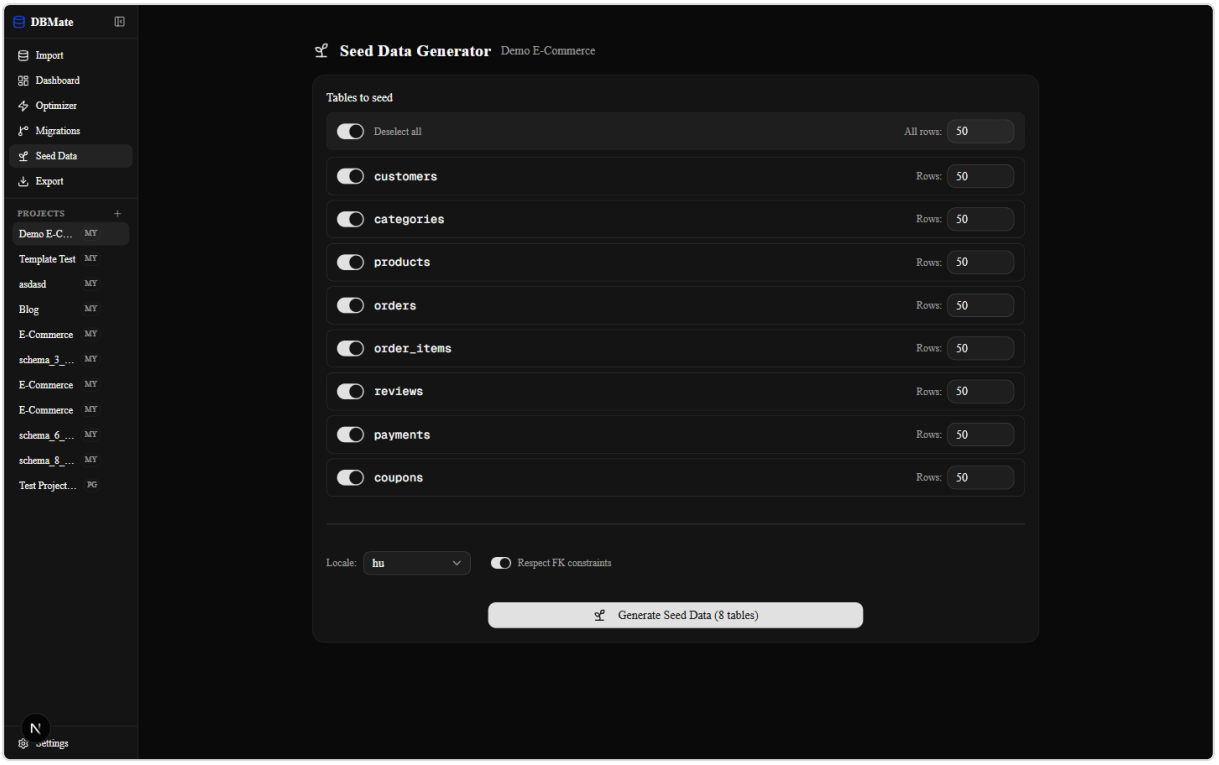
Migrations oldal — AI Migration Generator form



Generált migráció — V001, UP/DOWN SQL, Apply/Download gombok

4.8. Seed Data Generator

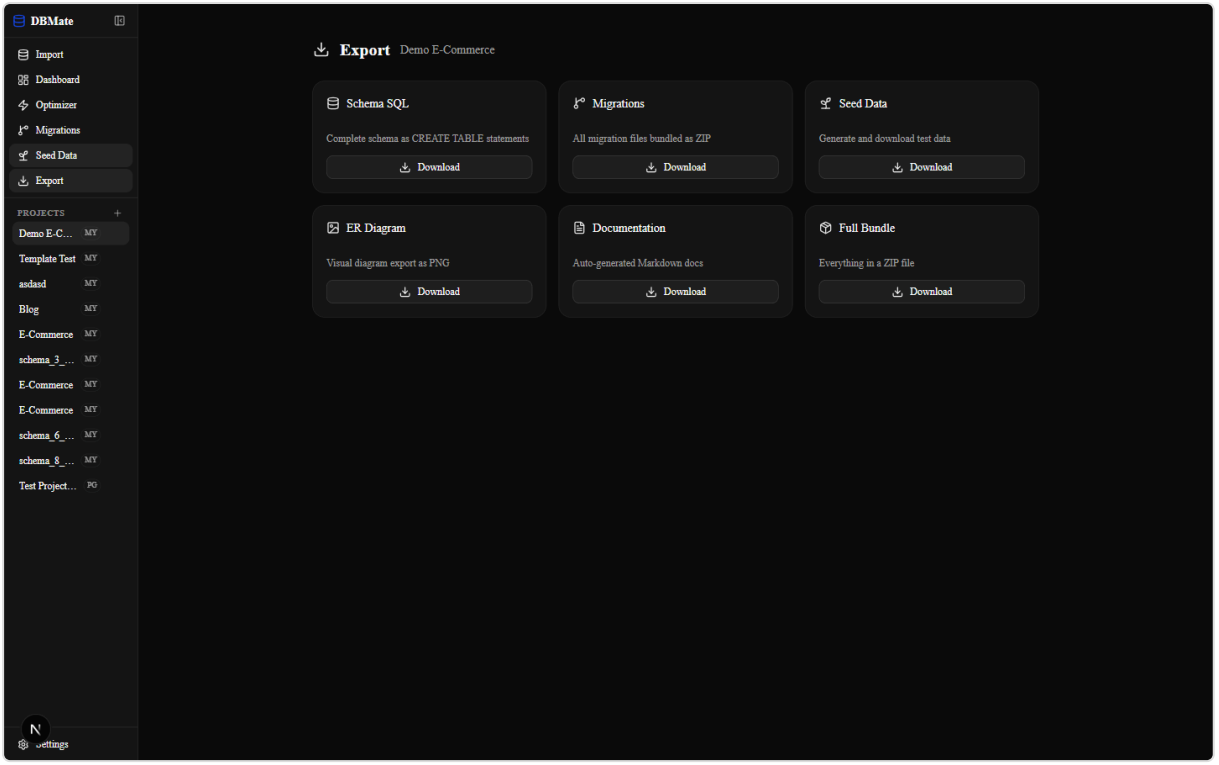
Teszadat-generálás AI segítségével, locale-tudatos (pl. magyar nevek, címek), FK-constraint tiszteletben tartásával, topológiai sorrendben.



Seed Data Generator — tábla-választó, sorlétszám, locale, FK respektálás

4.9. Export

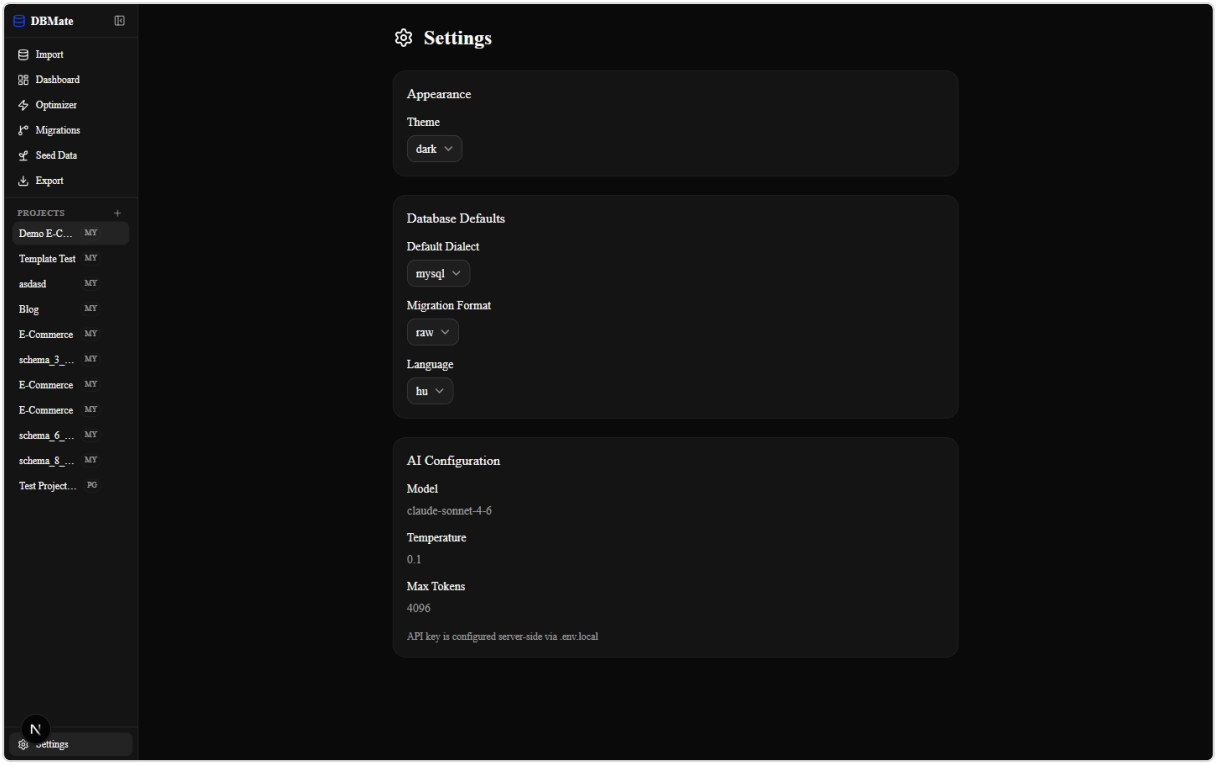
Többféle export formátum: Schema SQL, Migrations ZIP, Seed Data, ER Diagram PNG, auto-generált Markdown dokumentáció, és Full Bundle ZIP.



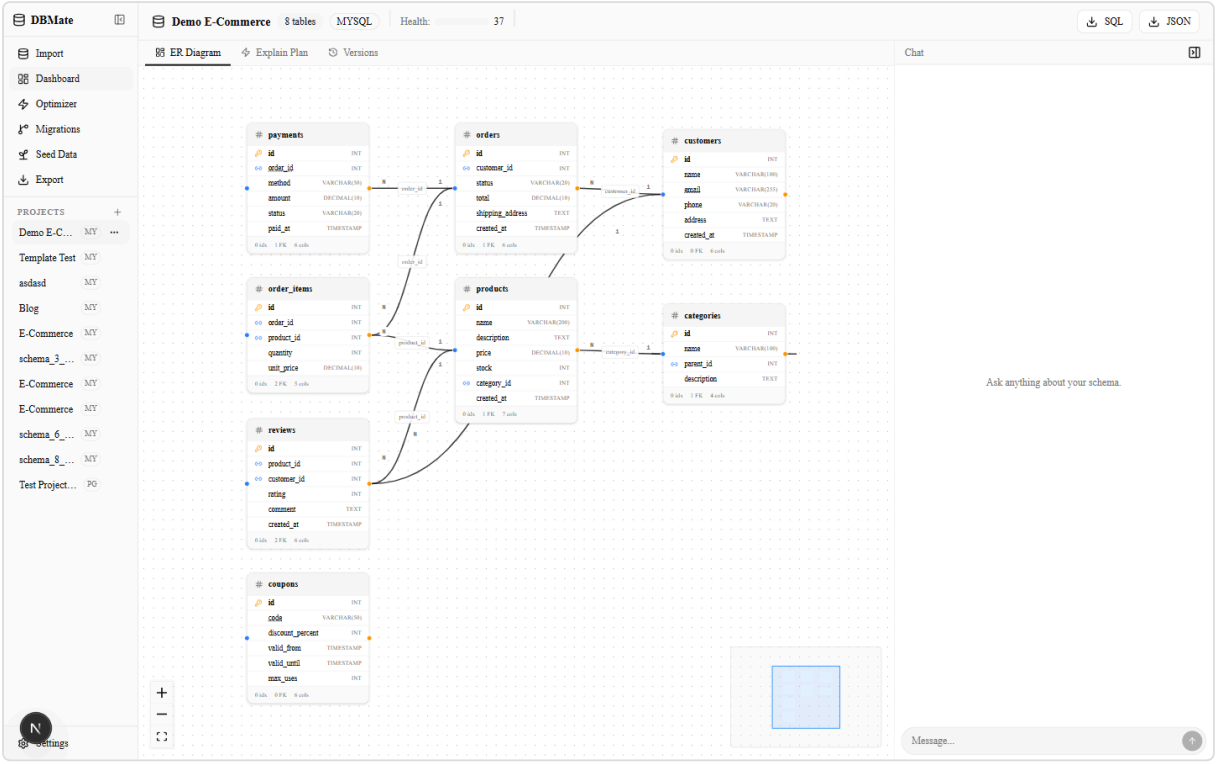
Export — 6 export formátum, egy kattintásos letöltés

4.10. Settings

Felhasználói beállítások: téma (dark/light/system), alapértelmezett dialektus, migrációs formátum, nyelv, AI konfiguráció. Az alkalmazás teljes dark és light módot támogat, amely a Settings oldalon váltható.



Settings — Appearance, Database Defaults, AI Configuration



Light mode — az alkalmazás teljes egészében támogatja a világos témát

5. Prompt Engineering technikák és tanulságok

5.1. System prompt tervezési mintázatok

Az alkalmazás 6 különálló AI endpointot használ, mindegyikhez dedikált system prompt tartozik (`src/lib/ai-prompts.ts`). A prompt-ok tervezése során a következő mintázatokat alkalmaztuk:

Persona + Scope constraint — Minden prompt egy szűk szakértői szerepet definiál, és expliciten korlátozza az elemzési területet. Például a normalizációs prompt: *„Analyze ONLY normalization quality. Do NOT check performance, security, or naming.*" Ez azért fontos, mert az AI hajlamos „túlszolgálni" — ha nem korlátozzuk, a normalizációs elemzés performance tanácsokat is ad, ami összekeveri a kategóriákat.

Output format kikényszerítés — Minden prompt tartalmazza: *„Return ONLY a JSON object (no markdown, no code blocks)*". Ez drámaian csökkentette a parse hibákat. A seed prompt esetében más a minta: *„Return ONLY raw SQL INSERT statements*" — mert ott nem JSON-t, hanem SQL-t várunk.

Negatív instrukciók — A chat prompt tartalmazza: *„NEVER invent tables/columns not in the schema*" és *„Admit uncertainty rather than hallucinate*". Ezek a negatív utasítások hatékonyabbnak bizonyultak, mint a pozitív megfogalmazások („Only use existing tables").

Kontextus tömörítés — A séma kontextust kompakt formátumban adjuk át az AI-nak (`buildSchemaContext()` függvény), nem a teljes CREATE TABLE SQL-t. Például: `TABLE orders: - id INT [PK, AUTO] - customer_id INT [NOT NULL] FK: customer_id -> customers(id) ON DELETE CASCADE`. Ez ~60%-kal kevesebb tokent használ, mint a nyers SQL.

5.2. Few-shot prompting

Minden AI végponthoz (normalization, migration, explain, seed) dedikált few-shot példákat definiáltunk — egy user-assistant üzenetpárt, amely pontosan a várt struktúrát mutatja be.

Példa a seed few-shot-ra (magyar locale):

```
User: "Schema: users(id INT PK AUTO, name VARCHAR(50)...) Config: 3 rows, locale: h
Assistant: "INSERT INTO users (id, name, email, status) VALUES
(1, 'Kovacs Istvan', 'kovacs.istvan@example.hu', 'active'),
(2, 'Nagy Katalin', 'nagy.katalin@example.hu', 'active'),
(3, 'Toth Peter', 'toth.peter@example.hu', 'inactive');"
```

A few-shot példák hatása mérhető volt: nélkülük az AI ~30%-ban rossz JSON-struktúrát adott vissza (pl. `up_sql` a várt `upSQL` helyett). A few-shot után ez ~5%-ra csökkent, a maradék hibákat a retry mechanizmus kezeli.

5.3. Retry + validáció stratégia

Az `withRetry()` függvény (`src/lib/ai-retry.ts`) egy általános retry wrapper, amely Zod sémával validálja az AI outputot:

1. **Hívás** → AI válasz → `extractJSON()` → Zod parse
2. **Ha sikertelen** → a Zod hibaüzenetet visszainjektáljuk a következő promptba: `[RETRY: Your previous response had errors: JSON validation failed: ...]`
3. **Temperature csökkentés** — minden retry-nál 0.05-tel csökken (pl. 0.1 → 0.05 → 0.0), hogy az AI determinisztikusabb legyen

Ez a „self-correcting" minta a legmegbízhatóbb módszer volt: az AI tipikusan a 2. próbálkozásra javítja a hibát, mert látja a konkrét Zod hibaüzenetet. Maximum 3 próbálkozás (1 + 2 retry) történik.

Érdekesség: a migrate endpointban inline key-normalizálást alkalmazunk — az AI néha `up_sql`-t ad `upSQL` helyett, ezért a Zod validáció előtt explicit mappinggel javítjuk a kulcsneveket (pl. `if ('up_sql' in json) json.upSQL = json.up_sql`).

5.4. Claude Code CLI mint fejlesztési eszköz

A teljes projekt **Claude Code** (Anthropic CLI) segítségével készült — nem webes Claude-dal, hanem a terminálból futtatott agenssel, amely közvetlenül szerkeszti a fájlokat, futtat parancsokat, és olvassa a kódot.

A fejlesztési workflow:

1. **Master Plan** megírása egy hosszú prompt-ban → ez adta az architektúrát
2. **Fázisonkénti implementáció** — minden fázishoz egy-egy konverzáció, pl. *„Implementáld a Phase 3-at a Master Plan alapján*"
3. **Review ciklus** — *„Nézd át az AI-hoz kapcsolódó route-okat és azonosítsd a javítási lehetőségeket*" → ez generálta az AI Improvement Plan-t (35 javítási pont, v6-ig iterálva)
4. **Iteratív javítás** — a review eredményei alapján újabb konverzációkban implementáltuk a javításokat

Tapasztalat: a Claude Code kiemelkedően hatékony volt a fázisonkénti fejlesztésre, mert a konverzáción belül megtartotta a kontextust. Viszont konverzációk között elveszett a kontextus — erre megoldás volt a Master Plan és az AI Improvement Plan, amelyek állandó referenciát biztosítottak.

5.5. Ami nehéz volt / kihívások

- **Proxy limitáció megkerülése** — Lásd a 2. fejezet részletes leírását. A legfontosabb tanulság: az LLM output mindig „piszkos" — érdemes defensíven parseolni. Az `extractJSON()` függvény (~80 sor) megbízhatóbban működik, mint bármilyen regex-alapú megoldás, mert brace-balancing-gel és csonkolt JSON javítással dolgozik.

- **Kontextus ablak menedzselés** — A legnagyobb kihívás az volt, hogy a Master Plan + az aktuális kód + a prompt együtt ne lépje túl a kontextus limitet. Megoldás: fázisonkénti prompt-ok, amelyek csak a releváns részt tartalmazzák, és a séma JSON kompakt formátumban.
- **AI „hallucinálás" kezelése** — Az AI időnként nem létező oszlopnevekre vagy függvényekre hivatkozott. A Zod validáció + retry mechanizmus ezt kezeli: ha a válasz nem valid, a hibaüzenetet visszaadjuk az AI-nak, amely tipikusan a második próbálkozásra javít.
- **Lokalizáció** — A magyar nyelvű válaszok generálása külön system prompt instrukciókat igényelt. Az AI hajlamos visszaváltani angolra, ha a séma angol neveket használ — a system prompt-ban explicit „Válaszolj magyarul" utasítás kellett, valamint a few-shot példában magyar neveket adtunk (pl. „Kovacs Istvan").
- **SSR hydration mismatch** — A Zustand persist middleware localStorage-ból tölt, ami eltér a szerveren renderelt alapértelmezéstől. Megoldás: `mounted` state pattern — a komponens először az alapértelmezett értékeket rendereli, és csak a kliens-oldali `useEffect` után váltja be a valós értékeket.

5.6. Architektúra döntések

- **Next.js API Routes az LLM proxy-ként** — A szerver oldali API route-ok kezelik az AI hívásokat, így az API kulcs nem kerül a kliensre. A streaming SSE-vel a chat válaszok real-time jelennek meg.
- **Hibrid statikus + AI elemzés** — Az optimizer nem kizárólag AI-ra támaszkodik: a statikus elemző (`src/lib/static-analyzer.ts`) azonnal felismeri a hiányzó indexeket, audit oszlopokat, naming convention sértéseket — míg a komplexebb normalizációs analízist az AI végzi. Ez gyorsabb és megbízhatóbb, mint ha mindent az AI-ra bíznánk.
- **SQLite mint lokális adatbázis** — Egyetlen fájl, nincs külső függőség, WAL mód a párhuzamos olvasáshoz. A sémák, verziók, migrációk, chat history mind itt tárolódnak.
- **Zustand állapotkezelés** — 3 store (schema, chat, settings) a persist middleware-rel, localStorage-ba mentve. Ez lehetővé teszi, hogy a felhasználó böngésző-frissítés után is a korábbi állapotban folytassa.

6. Tesztelés

A projekt átfogó tesztelésen esett át, három szinten:

6.1. Unit tesztek (Vitest)

48 tesztfájl, **535 teszt**, amely lefedi:

- **Lib/service modulok** (98% lefedettség) — `extractJSON`, `ai-retry`, `sql-parser`, `static-analyzer`, `validations`, `rate-limiter`, `migration-templates`, `seed-utils`, `seed-validator`, `doc-generator`, `api-helpers`, `apply-fix`, `batch-fix`, `db`
- **Zustand store-ok** (3/3) — `schema-store`, `chat-store`, `settings-store`
- **React komponensek** (21 fájl) — `ai-chat`, `sidebar`, `new-project-wizard`, `table-details`, `explain-plan`, `health-score-gauge`, `issue-card`, `migration-list`, `template-gallery`, `error-boundary`, stb.

```
pnpm test            # egyszer futtat
pnpm test:coverage   # lefedettség riporttal
```

6.2. E2E tesztek (Playwright)

12 tesztfájl, **70 teszt**, amely az alkalmazás teljes felhasználói folyamatait teszteli:

- **API route-ok** — `/api/health`, `/api/parse`, `/api/analyze`, `/api/schemas`, `/api/settings`, `/api/chat`
- **Felhasználói flow-k** — import flow (template → parse → dashboard), sidebar navigáció, dashboard ER diagram, optimizer elemzés, migrations generálás, export letöltés, settings mentés, chat küldés, séma verziók
- **Akadálymentesség (a11y)** — axe-core WCAG 2 AA szabvány ellenőrzés minden fő oldalon

```
pnpm test:e2e        # Playwright futtatás
```

6.3. Coverage riport

All files

60.17% Statements 1245/2069 **56.59%** Branches 807/1426 **54.16%** Functions 260/480 **60.59%** Lines 1107/1827

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
components/chat	31.15%	43/138	27.9%	24/86	60.71%	17/28	27.86%	34/122
components/dashboard	5.29%	18/340	10.94%	29/265	9.78%	9/92	5.92%	17/287
components/import	12.32%	9/73	6.25%	2/32	25%	5/20	13.23%	9/68
components/layout	18.44%	19/103	27.77%	15/54	20.51%	8/39	20%	19/95
components/migrations	34.78%	24/69	27.27%	15/55	30.43%	7/23	35.93%	23/64
components/optimizer	45.6%	83/182	49.26%	67/136	50.79%	32/63	44.65%	71/159
components/seed	55.31%	26/47	50%	9/18	42.85%	9/21	57.5%	23/40
components/shared	67.48%	137/203	74.21%	141/190	70.76%	46/65	66.66%	128/192
lib	96.93%	886/914	85.59%	505/590	98.44%	127/129	97.87%	783/800

Code coverage generated by istanbul at 2026-03-25T12:27:57.949Z

Coverage összesítés — lib 97%, components 5-67%, összesen 60.59% sorlefedettség

All files lib

96.93% Statements 886/914 **85.59%** Branches 505/590 **98.44%** Functions 127/129 **97.87%** Lines 783/800

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File	Statements		Branches		Functions		Lines	
ai-retry.ts	100%	13/13	75%	6/8	100%	2/2	100%	11/11
ai-service.ts	100%	67/67	85.45%	47/55	100%	13/13	100%	58/58
api-helpers.ts	100%	3/3	100%	3/3	100%	1/1	100%	3/3
apply-fix.ts	97.41%	301/309	89.61%	164/183	97.87%	46/47	98.51%	266/270
batch-fix.ts	96.29%	26/27	77.77%	21/27	100%	6/6	96%	24/25
doc-generator.ts	100%	54/54	90%	18/20	100%	1/1	100%	54/54
extract-json.ts	97.36%	74/76	95.45%	42/44	100%	1/1	97.82%	45/46
migration-formatter.ts	100%	5/5	100%	4/4	100%	1/1	100%	5/5
migration-templates.ts	100%	34/34	94.59%	35/37	100%	11/11	100%	33/33
rate-limiter.ts	93.75%	15/16	90%	9/10	100%	1/1	100%	15/15
seed-utils.ts	100%	45/45	95.45%	21/22	100%	7/7	100%	36/36
seed-validator.ts	100%	25/25	100%	11/11	100%	5/5	100%	24/24
sql-parser.ts	87.6%	106/121	68.37%	80/117	100%	14/14	90.65%	97/107
static-analyzer.ts	100%	87/87	89.36%	42/47	100%	15/15	100%	81/81
utils.ts	100%	2/2	100%	0/0	100%	2/2	100%	2/2
validations.ts	96.66%	29/30	100%	2/2	50%	1/2	96.66%	29/30

Code coverage generated by istanbul at 2026-03-25T12:27:57.949Z

Lib modulok részletes lefedettség — 16 fájlból 12 darab 100% statement coverage

Miért alacsony egyes komponensek lefedettsége?

Az alacsony coverage-ű komponensek (pl. `components/dashboard` 5.92%, `components/import` 13.23%) tudatos döntés eredménye. Ezek komplex, interaktív UI elemek (ER diagram a

`@xyflow/react`-tel, CodeMirror SQL szerkesztő, drag & drop), amelyeknél:

- A unit tesztelés ROI nagyon alacsony — a renderelés külső könyvtárak belső állapotától függ (canvas, virtualizált DOM)
- Az ER diagram (`er-diagram.tsx`, 261 sor) egy `@xyflow/react` wrapper, amelynek a belső layout algoritmusát nem érdemes unit tesztelni
- Ezeket az **E2E tesztek** fedik le hatékonyabban (Playwright böngészőben teszteli a teljes flow-t: import → dashboard megjelenés → tábla kattintás)

Ezzel szemben a **lib modulok 97.87%-os lefedettséget** értek el, mert ezek az üzleti logikát tartalmazzák (SQL parsing, AI válasz feldolgozás, validáció, statikus elemzés) — itt a legmagasabb a hibakockázat, és a unit tesztelés a leghatékonyabb.

6.4. Teszt összefoglaló

Típus	Keretrendszer	Fájlok	Tesztek
Unit	Vitest + Testing Library	48	535
E2E	Playwright	12	70
A11y	axe-core (WCAG 2 AA)	1	7
Összesen		**61**	**605**

Coverage	Statements	Branches	Functions	Lines
Lib modulok	96.93%	85.59%	98.44%	**97.87%**
Összesen	60.17%	56.59%	54.16%	**60.59%**

7. Összefoglaló

Metrika	Érték
Összes kódsor	**20 556** TypeScript/TSX
Alkalmazás kód	13 555 sor (~100 fájl)
Teszt kód	7 001 sor (60 fájl)
API végpontok	15
UI komponensek	60+
AI funkciók	6 (chat, analyze, migrate, seed, explain, index-analysis)
Tesztek	**605** (535 unit + 70 E2E)

Metrika	Érték
Lib coverage	**97.87%** lines
Összesített coverage	60.59% lines
Build status	Sikeres (0 hiba)